

BulsuScholar

# Revised System Implementation and Acceptance Guide

---

Coverage: SemiFinalRevisions Numbers 1, 4, 5, 6, 7, 8, 9, and 10

Prepared from the implementation register and production deployment checklist

## Current release status

Implemented and locally verified, but not yet production-complete. Keep Maintenance Mode enabled until database, Auth, email, Storage, cron, role, and production smoke-test gates pass.

Prepared: September 28, 2026

Named acceptance testers: Johnvher (Admin), Veejay (Grantor), Emmerson (Student)

## Source documents

- docs/SemiFinalRevisions.md
- docs/deployment/DeploymentChecklist.md
- docs/UIReleaseAudit.md

# 1. Executive Summary: What Changed

The revision moves sensitive decisions from browser-side table access to authenticated backend workflows. It also adds manual account approval, reusable student profile revisions, document review, authoritative roster assignment, grantor scope, targeted announcements, FIFO waitlists, inactivity email verification, private recovery, student history, Signed SOE submission, and formal release gates.

| No. | Area                       | Main change                                                                                                                              | Current status                                                     |
|-----|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 1   | Security and accounts      | Restricted direct data access; backend login, logout/recovery, pending signup approval, role ownership checks.                           | Implemented locally; production/staging role tests remain.         |
| 4   | Profile and documents      | Editable signed profile revisions, private versioned documents, independent review, COR policy, first-year first-semester ROG exemption. | Implemented locally; private Storage and live review need staging. |
| 5   | Scholarship and roster     | Server preview/commit import, exact scholarship matching, atomic roster conversion/conflict handling, protected Request Materials.       | Implemented locally; transaction/concurrency tests remain.         |
| 6   | Scope and reports          | Grantor classification/location scope, filtered reports, audit records, student-number correction.                                       | Implemented; official point formula remains blocked.               |
| 7   | Announcements and waitlist | Audience preview, deduplicated notices, global FIFO queue, 24-hour offers, cron expiry.                                                  | Implemented; Railway cron and live queue tests remain.             |
| 8   | OTP and sessions           | 30-day inactivity email verification, verified sessions, cross-tab identity, lost-email recovery conversation.                           | Implemented; live Brevo/Auth/session tests remain.                 |
| 9   | History and Signed SOE     | Append-only student history; Signed SOE upload before Finish; admin reason-required reopen.                                              | Implemented; private upload and cycle tests remain.                |
| 10  | Release acceptance         | UI audit, automated checks, deployment order, smoke-test and rollback gates.                                                             | Local checks pass; production acceptance is blocked.               |

## Important policy limitation

The recommendation score is informational and is labelled Recommendation Score - Not Official Ranking. The official scholarship point formula has not been supplied and must not be treated as implemented.

# 2. What Was Implemented

## Number 1 - Security, approval, and authentication

- Revoked broad direct access to sensitive tables and private Storage; legacy reads use authenticated portal compatibility endpoints with role and ownership filtering.
- New signups remain pending after email confirmation when manual approval applies. Full admins accept or reject; pending users cannot start an active portal session.
- Unified backend login records failed attempts, locks accounts at the configured limit, resets failures after successful recovery, and keeps administrator unlock under root control.
- Student/grantor password recovery resolves email server-side and returns generic responses. Grantor Auth migration tooling exists and the validated production grantor migration was reported complete.
- Signed-SOE access now requires Requirements permission or full-admin authority.

## Number 4 - Student profile and document verification

- Students edit a reusable master profile draft, save it, sign each submitted revision, preview/export the official PDF, and retain immutable revision history.
- Permanent address is required; current address is optional. Scholarship-owned grantor fields are populated from the selected application and are not student-editable.
- COR, ROG, identity, profile, signatures, decisions, and generated PDFs are versioned and private. Rejected documents retain the earlier version and require a correction reason.
- First-year, first-semester students are automatically exempt from ROG without a fake approval record. ROG becomes required in later semesters or years.
- Document Review allows full admins and student reviewers to approve or reject submissions independently from account approval. A failed earlier requirement pauses later tracking and a corrected approval restores preserved progress.

## Number 5 - Scholarship and roster workflow

- Grantor/admin roster imports use Preview then Commit. The backend validates ownership, exact recognized scholarship, duplicates, account matches, identity mismatches, commitments, and active-roster conflicts.
- A matching ordinary application is converted in place, preserving its ID, tracking, documents, and history while releasing its public slot.
- Authoritative roster awards do not consume public slots and do not fabricate uploads, material requests, downloads, signatures, or SOE files.
- A unique valid roster match assigns the scholarship after account approval. Conflicts assign nothing until a full admin verifies identity and chooses one row.
- Request Materials is an atomic backend operation that accepts only the application ID and returns precise rejection codes instead of a generic eligibility error.

## Number 6 - Scope, filtering, reports, and correction

- Grantors have a separate Government, Private, or Others classification and an optional versioned municipality/city scope configured by a full admin.
- New application and waitlist decisions snapshot the self-declared permanent address and scope version, so later scope changes do not rewrite existing decisions.
- Grantor applicant lists support server-side status, location, scholarship, cycle, document, and tracking filters. CSV/PDF exports are generated from backend-fetched records and audited.
- Full admins can correct a student number with uniqueness checks and an audit reason while preserving the Auth UUID and linked history. Existing sessions are revoked.
- The existing recommendation algorithm remains informational only; no official ranking formula was invented.

## Number 7 - Announcements, notifications, and FIFO waitlist

- Admin/grantor announcement publishing begins with a server-owned audience preview that expires after 15 minutes. Grantors can target only their connected students.
- Publishing uses immutable recipient snapshots and deterministic delivery IDs to prevent duplicate inbox/dashboard notices during retries.
- The root-controlled global waitlist capacity defaults to zero. When enabled, queued plus offered records count toward the global cap.
- Each scholarship queue is FIFO. A released or added slot becomes an exclusive 24-hour offer for the first eligible student; accept creates an ordinary application, while decline/expiry advances the queue.
- A protected Railway cron worker calls the expiry endpoint every five minutes using CRON\_SECRET.

## Number 8 - Email verification, sessions, and lost email

- After a successful password check, students/grantors inactive for 30 days receive a six-digit email challenge before any Supabase session is returned.
- Codes are hashed, expire after 10 minutes, allow three wrong attempts, enforce 60-second resend delay, and limit delivery to five sends per hour.
- Verified sessions bind Auth UUID, JWT session ID, portal identity, lock state, and sessionValidAfter. Protected backend routes validate all of these.
- Normal tabs share login/logout through local storage, BroadcastChannel, and storage events. Incognito and other devices remain independent.
- Lost-email recovery uses a signed-out private support conversation, secret capability URL, private evidence attachments, root review, and confirmation of the proposed mailbox before Auth/profile email changes.

## Number 9 - Student history and Signed SOE

- The private append-only history ledger records deterministic student-safe application, document, tracking, roster, waitlist, commitment, cooldown, withdrawal, and cycle events.
- Students have a History page with pagination, cycle/type filters, and safe related-page links; staff-only notes and security metadata are excluded.
- Ordinary committed scholars upload Signed SOE after office signing. PDF/PNG/JPEG up to 10 MB is accepted; successful submission immediately marks Submitted and completes Finish.
- The upload proves submission only, not staff verification or physical custody. Authoritative roster scholars are exempt.
- Authorized admins can preview and reopen Signed SOE with a mandatory reason. The original remains immutable and the student submits a new version.

## Number 10 - Interface and release acceptance

- Local UI checks covered public/authentication, Student and Grantor desktop/mobile, and Admin desktop routes and conditional states.
- Responsive, dark-mode, Quick Actions, modal focus/Escape, icon-label, preview layering, and Admin Profile runtime defects were corrected.
- Local verification passes 136 backend tests, six frontend security checks, ESLint, production build, backend compilation, and offline migration parsing.
- Production remains blocked until staging database, Auth, Brevo, Storage, cron, role, concurrency, rollback, and production smoke-test evidence exists.

### 3. Implemented Flows

#### Flow A - Student account creation and approval

Student submits signup -> Supabase sends confirmation email

|  
v

Student confirms email -> account remains Pending

|  
v

DECISION: Manual approval required? YES -> Full admin reviews; NO -> activation policy continues

|  
v

DECISION: Admin accepts? YES -> activate account + Welcome email; NO -> Rejected email + cleanup/restart path

|  
v

Activated student logs in -> verified portal session -> Student Dashboard

## Flow B - Login, logout, and 30-day email verification

User ID + password -> backend resolves account without exposing account data

|  
v

DECISION: Disabled or blocked? YES -> deny login; NO -> Supabase Auth password check

|  
v

DECISION: Password failed? YES -> increment failures -> limit reached? lock account

|  
v

DECISION: Inactive for at least 30 days? NO -> create verified session; YES -> send six-digit email code

|  
v

Correct unexpired code -> reauthenticate -> create verified session -> dashboard

|  
v

Logout in one normal tab -> BroadcastChannel/storage event -> other normal tabs log out

## Flow C - Profile and document review

Student edits profile draft -> Save Draft -> Preview

|  
v

Student provides fresh signature -> Submit Revision -> immutable PDF/version

|  
v

Student uploads current-cycle COR + identity + applicable ROG

|  
v

DECISION: First year AND first semester? YES -> ROG not required; NO -> ROG required

|  
v

Reviewer checks each current version -> Approve or Reject with reason

|  
v

DECISION: All applicable requirements approved? YES -> complete Document Review; NO -> show correction action

|  
v

Corrected version approved -> restore preserved tracking stage

## Flow D - Authoritative roster import and assignment

Grantor/Admin maps spreadsheet -> Preview batch

|  
v

Backend validates grantor ownership + exact scholarship + identity + duplicates + commitments

|  
v

DECISION: Blocking row exists? YES -> Commit disabled; correct rows and preview again

|  
v

Commit valid stored batch -> transaction/advisory lock -> reuse or create roster/application records

|  
v

DECISION: Unique valid active match? YES -> authoritative assignment; NO -> visible conflict, assign nothing

|  
v

Full admin verifies conflict in person -> selects one roster -> alternatives archived without cooldown

|  
v

Applicable approved documents complete -> system completes roster tracking through Finish

## Flow E - Ordinary application and Request Materials

Student applies to one or more eligible grantor scholarships

|  
v

Application reserves one public slot -> documents/tracking continue

|  
v

Student selects Request Materials -> backend receives application ID only

|  
v

DECISION: Ownership, eligibility, documents, versions, requirements, slot, conflict, and cooldown valid?

|  
v

NO -> return precise reason; make no partial slot/application/request change

|  
v

YES -> commit selected scholarship -> create/retry request -> close alternatives -> release their slots

|  
v

Staff approves materials -> student downloads/signs -> proceeds to Signed SOE

## Flow F - Full scholarship and FIFO waitlist

Student attempts eligible application -> no public slot remains

|  
v

DECISION: Global waitlist capacity greater than zero and not full? NO -> explain unavailable; YES -> join queue

|  
v

Queue position = queued\_at then stable ID

|  
v

Slot released or added -> transaction reserves it for first still-eligible queued student

|  
v

Student receives inbox/dashboard 24-hour offer

|  
v

DECISION: Accept before expiry? YES -> ordinary active application; NO/Expired -> offer next eligible student

|  
v

Scholarship commitment elsewhere -> close all competing queues/offers and restore reserved slots

## Flow G - Targeted announcement

Admin/Grantor chooses permitted target filters

|  
v

Backend resolves recipients -> immutable preview snapshot (15 minutes)

|  
v

DECISION: Preview valid and unexpired? NO -> regenerate; YES -> publish from preview ID

|  
v

Transaction creates deterministic student deliveries

|  
v

Student sees one inbox/dashboard notice -> direct link opens the correct announcement or action

## Flow H - Signed SOE, Finish, and renewal

Ordinary committed scholar completes office-signing stage

|  
v

Upload Signed SOE -> backend validates owner, cycle, stage, type, size, and duplicate status

|  
v

Valid upload -> immutable private version -> status Submitted -> Finish complete

|  
v

Student sees next-semester renewal message + history event

|  
v

DECISION: Admin reopens for unreadable/wrong file? YES -> reason required -> return to Upload Signed SOE

|  
v

New cycle -> preserve completed history -> restart applicable document requirements

**Flow I - Safe deployment**

Enable Maintenance Mode -> verify normal portals are blocked

|  
v

Create recoverable database backup + separate Storage copy/manifest

|  
v

Run local tests -> verify staged files and commit -> push one reviewed commit

|  
v

Apply only missing migrations on staging -> database compatibility first

|  
v

Deploy backend -> verify health/OpenAPI -> configure cron and secrets

|  
v

Deploy frontend from same commit -> verify production domain

|  
v

Run Johnvher + Veejay + Emmerson acceptance suites

|  
v

DECISION: Any release blocker? YES -> keep/restore Maintenance Mode; NO -> reopen and record sign-off

## 4. What You Need to Check

### Security and accounts

- [ ] Anonymous and ordinary browser clients cannot read/write sensitive tables, call privileged RPCs, subscribe to private Realtime data, or open private files.
- [ ] Pending students cannot log in; only full admins approve/reject; Welcome/Rejected emails arrive once and match the final committed decision.
- [ ] Three failed logins lock the configured test account; recovery unlocks students/grantors only after password change; root alone unblocks admins.
- [ ] Normal-tab login/logout is shared; Incognito/second device remains independent; stale or revoked sessions fail protected backend calls.

## Profiles and documents

- [ ] Draft save/reopen, fresh signature, immutable revision, one-page PDF, private preview, permanent/current address behavior, and revision history.
- [ ] First-year first-semester ROG exemption; all other year/semester combinations require ROG.
- [ ] COR policy modes and exceptions; first-year alternative photo ID; upper-year Student ID/lost-card exception; scanner success never auto-approves.
- [ ] Rejection reason, corrected upload, compliance pause, preserved later steps, and automatic restoration after approval.

## Roster, applications, and materials

- [ ] Malformed, duplicate, unknown, ambiguous, identity-mismatch, commitment-conflict, repeated, and concurrent roster imports.
- [ ] Matching application ID/history/documents are preserved and its public slot is released during authoritative conversion.
- [ ] Roster lock prevents withdrawal/competing application until individual archival; individual archive starts the 24-hour cooldown; grantor archive transfers servicing to admin.
- [ ] Request Materials succeeds only after all checks and never leaves partial state after a precise rejection or duplicate click.

## Scope, reports, announcements, and waitlist

- [ ] Inside/outside municipality behavior, versioned scope snapshot, self-declared-address disclosure, and cross-grantor denial.
- [ ] Filtered CSV/PDF report totals and immutable report audit; student-number correction preserves history/Auth UUID and revokes sessions.
- [ ] Audience preview counts, expiry, ownership restrictions, deduplicated publish retries, unread counts, direct links, and archived-user handling.
- [ ] Waitlist disabled/enabled, global cap, FIFO ordering, simultaneous joins, reserved 24-hour offer, accept/decline/expiry/rejoin, and slot restoration.

## History, SOE, and UI

- [ ] History ordering, pagination, cycle/type filters, safe descriptions, no staff-only notes, and correct related-page links.
- [ ] Signed SOE type/size/owner/stage/cycle checks, immediate Finish, reason-required reopen, retained earlier version, and roster exemption.
- [ ] Desktop/mobile, light/dark, empty/loading/error/disabled/conflict/archived states; no clipping, overflow, hidden focus, duplicate toast, modal-layer error, broken image, console error, or failed request.

## 5. Current Phase to Production: Step-by-Step Runbook

### STOP rule

If a migration result, backup, deployment commit, health check, email, private file, or core role flow is unclear or fails, stop. Do not rerun every SQL file, rerun the completed grantor Auth migration, force-push, or disable Maintenance Mode as a workaround.

### Phase map

#### From the current checkout to the live revised system

Current local working tree -> review every modified and untracked file

|  
v

Local release candidate -> automated checks pass -> one reviewed Git commit

|  
v

Staging -> Supabase migrations/config -> Railway API/cron -> Vercel preview

|  
v

Staging acceptance -> role, email, Storage, security, workflow, and UI evidence

|  
v

Production protected by Maintenance Mode -> backups and baseline recorded

|  
v

Production database compatibility -> Railway backend/cron -> Vercel frontend

|  
v

Controlled production smoke test -> release sign-off -> Maintenance Mode off

|  
v

DECISION: Any failed gate? YES -> stop/rollback and keep Maintenance Mode on; NO -> monitor

**Meaning of the three builds**

Development build is the code running on your computer. Release candidate is one reviewed Git commit that passed local checks and staging. Deployment build is the exact same commit built by Railway and Vercel with production environment variables. Do not edit files between staging approval and production deployment; corrections require a new commit and a new staging pass.

**Phase 1 - Turn the current checkout into a release candidate**

- [ ] Open VS Code -> Source Control. Review every modified and untracked file. Remove accidental local output, but do not discard work you do not understand.
- [ ] Confirm .env, API keys, service-role keys, database URLs, backup folders, and dist are not staged. Secrets belong in hosting dashboards, never Git.
- [ ] In PowerShell at the repository root, run: `git status --short; git diff --check; git diff --stat`.
- [ ] Run: `python -m compileall backend -q`.
- [ ] Run: `python -m unittest discover -s backend/tests -p test_*.py -v`.
- [ ] Run: `npm.cmd run test:frontend; npm.cmd run lint; npm.cmd run build`.
- [ ] Fix every error. A Vite large-chunk warning may be recorded, but a failed command blocks release.
- [ ] Stage only reviewed release files. Inspect with: `git diff --cached --name-only` and `git diff --cached --check`.
- [ ] Create one release commit and push it to GitHub. Record the full SHA using: `git rev-parse HEAD`. Do not use `npm run tapos` until the staged content has been reviewed because it automatically adds, commits, and pushes.
- [ ] Gate: GitHub must show the intended commit on the deployment branch, and the local working tree must contain no forgotten release file.

**Phase 2 - Prepare a safe staging environment**

- [ ] Use a separate staging Supabase project and staging test data. Do not rehearse destructive workflows against production users.
- [ ] Create or select a Railway staging environment/service and a Vercel Preview deployment connected to the release commit.
- [ ] Choose staging URLs, for example a Railway staging API URL and the Vercel Preview URL. Add both to the appropriate CORS/Auth redirect configuration.
- [ ] Create disposable student, grantor, full-admin, reviewer, limited-admin, and root fixtures plus a dedicated test mailbox.
- [ ] Record the staging project reference, release SHA, starting row counts, application IDs, queue positions, and expected notification counts.
- [ ] Gate: staging must be isolated enough that account locks, rejection, archival, deletion, waitlist expiry, and recovery tests cannot damage production records.

**Phase 3 - Configure and migrate staging Supabase**

- [ ] In Supabase Dashboard -> Project Settings -> API, copy the staging Project URL and publishable/legacy anon key for the frontend. Copy the service-role key only into trusted Railway/local secret storage.
- [ ] In Supabase -> Database -> Backups, confirm staging has a recovery point before migration testing. Database backups do not contain Storage object contents.
- [ ] In Supabase -> Storage, confirm the bulsuscholar bucket exists and is private. Copy any needed staging fixtures into it; never make private student documents public.
- [ ] In Supabase -> Authentication -> URL Configuration, set the staging Site URL and add the Vercel Preview callback/reset URL. Production URLs are configured separately in Phase 9.
- [ ] Open SQL Editor -> New query. For each migration below, first verify whether its objects/history already exist, then paste the entire file and click Run once. Write down filename, time, and result.

- [ ] 20260918120000\_preserve\_tracking\_during\_document\_compliance.sql
- [ ] 20260918123000\_add\_tracking\_finish\_stage.sql
- [ ] 20260918130000\_automatic\_roster\_and\_login\_security.sql
- [ ] 20260922170000\_revoke\_unsafe\_direct\_access.sql
- [ ] 20260922180000\_require\_manual\_student\_account\_approval.sql
- [ ] 20260923120000\_student\_profile\_document\_verification.sql
- [ ] 20260923124643\_scholarship\_roster\_workflow.sql
- [ ] 20260924024322\_scope\_reporting\_announcements\_waitlist.sql
- [ ] 20260924125213\_security\_history\_signed\_soe.sql
- [ ] 20260925103000\_complete\_student\_history\_ledger.sql
- [ ] If a query errors or times out, stop and inspect what committed before retrying. Do not blindly use db push or migration repair when migration history differs.
- [ ] Run Supabase Security Advisor and Performance Advisor. Confirm protected tables have RLS and that anon/authenticated users have no broad table privileges or privileged RPC execution.
- [ ] Use an anonymous REST request and an ordinary authenticated test account to prove sensitive reads, writes, deletes, Realtime subscriptions, privileged RPCs, and private Storage access are denied.
- [ ] Do not rerun the completed production grantor Auth migration. Only a deliberately isolated staging fixture may use the dry-run migration workflow.
- [ ] Gate: all expected tables/functions exist, advisor findings are reviewed, private files remain private, and transaction/authorization tests pass.

## Phase 4 - Configure Brevo and Supabase Auth email

- [ ] In Brevo -> Senders, Domains & Dedicated IPs, verify the sender/domain used by no-reply@bulsus scholar.com. Complete SPF/DKIM steps shown by Brevo before relying on delivery.
- [ ] In Brevo -> SMTP & API -> API Keys, create or select a restricted production API key for the Railway backend. Store it as BREVO\_API\_KEY in Railway only.
- [ ] In Brevo -> SMTP & API -> SMTP, obtain the SMTP login and SMTP key. The SMTP key is not BREVO\_API\_KEY.
- [ ] In Supabase -> Authentication -> SMTP Settings, enable Custom SMTP and enter the Brevo SMTP host/port/login/key plus the verified sender.
- [ ] In Supabase -> Authentication -> Email Templates, review Confirm signup and Reset password links against the configured Site URL and redirect allow-list.
- [ ] Use the staging mailbox to test confirmation, Welcome, rejection, inactivity code, password recovery, and proposed-new-email confirmation. Each expected message must arrive once and link to staging.
- [ ] Check Brevo transactional logs for delivered, deferred, bounced, or blocked status. A database notification row does not prove email delivery.
- [ ] Gate: Supabase Auth mail and backend API mail both arrive from the correct sender without exposing secrets or linking to production during staging.

## Phase 5 - Deploy the staging Railway API and worker

- [ ] In Railway, create/select the staging environment and deploy the exact release SHA using the repository Dockerfile. Do not redeploy an older artifact.
- [ ] In the API service -> Variables, set SUPABASE\_URL, SUPABASE\_SERVICE\_ROLE\_KEY, SUPABASE\_STORAGE\_BUCKET=bulsus scholar, FRONTEND\_URL, and DOCUMENT\_SCAN\_ALLOWED\_ORIGINS to staging values.

- [ ] Set EMAIL\_PROVIDER=brevo, BREVO\_API\_KEY, BREVO\_SENDER\_NAME, BREVO\_SENDER\_EMAIL, and BREVO\_REPLY\_TO\_EMAIL.
- [ ] Generate different random values of at least 32 characters for ROOT\_SESSION\_SECRET, PORTAL\_EMAIL\_CODE\_SECRET, PUBLIC\_RECOVERY\_CODE\_SECRET, and CRON\_SECRET. Keep all four on Railway only.
- [ ] Set WEB\_CONCURRENCY=1 and UVICORN\_KEEP\_ALIVE=30 unless the verified production service intentionally differs. Keep DOCUMENT\_SCAN\_ALLOWED\_ORIGIN\_REGEX blank; never use https://\*.com.
- [ ] Optional root integrations such as ROOT\_DATABASE\_URL, Railway/Vercel tokens, and deploy hooks should remain unset unless the feature is deliberately configured with least privilege.
- [ ] Wait for Railway status Success. Inspect build and runtime logs for import, migration compatibility, CORS, secret, scanner, or startup errors.
- [ ] Open /health, /deployment/health, /scan-document/health, /email/health, and /openapi.json on the staging API. Required routes must be present before frontend testing.
- [ ] Create a second Railway service named waitlist-expiry from the same commit. Set BACKEND\_API\_URL to the staging API and CRON\_SECRET to the exact API value; Start Command: python -m backend.waitlist\_expiry\_job; schedule: \*/5 \* \* \* \*.
- [ ] Run the worker once and confirm its log contains ok=true. A 401, 403, timeout, or missing-secret result blocks release.
- [ ] Gate: API health is ok, deployment health reports required configuration, OpenAPI contains revised routes, CORS allows only intended frontend origins, and the cron worker succeeds.

## Phase 6 - Deploy the staging Vercel frontend

- [ ] In Vercel -> Project -> Settings -> Environment Variables, add Preview values for VITE\_SUPABASE\_URL, VITE\_SUPABASE\_ANON\_KEY, VITE\_SUPABASE\_STORAGE\_BUCKET, VITE\_APP\_URL, VITE\_PUBLIC\_SITE\_URL, VITE\_BACKEND\_API\_URL, and VITE\_DOCUMENT\_SCAN\_API\_URL.
- [ ] VITE\_APP\_URL/VITE\_PUBLIC\_SITE\_URL must be the staging Preview/custom URL; backend variables must point to the staging Railway API.
- [ ] Never add SUPABASE\_SERVICE\_ROLE\_KEY, BREVO\_API\_KEY, database URLs, ROOT\_SESSION\_SECRET, PORTAL\_EMAIL\_CODE\_SECRET, PUBLIC\_RECOVERY\_CODE\_SECRET, or CRON\_SECRET to Vercel or any VITE\_ variable.
- [ ] Deploy the exact same release SHA used by Railway. Confirm Vercel reports Ready and inspect build logs for missing-variable or bundle errors.
- [ ] Open the Preview URL in a private browser, verify its API requests go to staging, and check browser console/network logs for failed requests, CORS errors, mixed environments, or exposed secrets.
- [ ] Gate: the staging frontend and backend report the same SHA and use only staging Supabase/data/mail fixtures.

## Phase 7 - Approve the release candidate in staging

- [ ] Run the Johnvher, Veejay, and Emmerson suites in Sections 6-8 and the consolidated matrix in Section 11. Run root checks for waitlist capacity, admin unblock, recovery review, audit views, and destructive confirmations.
- [ ] Verify student/grantor at desktop and mobile sizes, Admin/root at desktop, and all required light/dark, empty, loading, error, disabled, archived, conflict, modal, toast, and file-preview states.
- [ ] Verify database state, immutable history/audit rows, notifications, email delivery, private file access, and navigation after every destructive workflow.
- [ ] Exercise duplicate clicks, retries, concurrent Request Materials/import/waitlist operations, expired previews/offers/codes, and session revocation.
- [ ] Log every defect with role, route, fixture, steps, expected/actual result, screenshot, console/network evidence, severity, fix commit, and retest result.
- [ ] Any correction creates a new commit. Redeploy that SHA to staging and rerun the affected test plus regression checks; never patch production separately.

- [ ] Gate: no unresolved critical/high issue, all release blockers pass, and Johnvher/Veejay/Emmerson plus release owner sign the approved SHA.

## Phase 8 - Protect and baseline production

- [ ] Open <https://bulsuscholar.com/root> -> Settings -> Portal controls. Turn Maintenance Mode on, save, confirm, and prove normal student/grantor/admin portals are blocked while root remains available.
- [ ] In production Supabase -> Database -> Backups, confirm a usable restore point and record its time. Stop if there is no recoverable database backup.
- [ ] Run the Storage backup script from a trusted machine with production service-role credentials and retain the object manifest/checksums separately. Database backup does not include Storage objects.
- [ ] Record the current production Supabase project, migration/object state, Railway/Vercel deployment IDs and SHAs, important row counts, domain/DNS state, environment status, and rollback owners.
- [ ] Confirm the approved release SHA is immutable on GitHub and no new untested commit has replaced it.
- [ ] Gate: Maintenance Mode is visibly active, both data and file recovery evidence exist, and the previous working deployment identifiers are recorded.

## Phase 9 - Configure production Supabase and Brevo

- [ ] Compare production database objects/history with the ten migration files. Apply only missing migrations in dependency order through SQL Editor, one file at a time, recording result and time.
- [ ] After each migration, stop on any error or unclear result. Do not immediately rerun it because earlier statements may already have committed.
- [ ] In production Supabase Authentication -> URL Configuration, set Site URL to <https://bulsuscholar.com> and allow the exact confirmation/reset callback URLs required by the application.
- [ ] Confirm Custom SMTP uses the verified Brevo sender and SMTP key. Confirm templates link to production, not staging.
- [ ] Confirm the bulsuscholar Storage bucket remains private, RLS is enabled for exposed-schema tables, protected functions are not executable by PUBLIC/anon/authenticated, and advisors are reviewed.
- [ ] Run readiness and anonymous-denial checks before deploying code. Do not rerun `migrate:grantor-auth -- --execute`; the validated production grantor migration was already completed.
- [ ] Gate: production schema is compatible with the release while Maintenance Mode remains on, and no unauthorized direct access is possible.

## Phase 10 - Deploy the production Railway backend and cron

- [ ] In the Railway production API service, verify production values for all Phase 5 variables. `FRONTEND_URL` and allowed origin must be <https://bulsuscholar.com>; `BACKEND_API_URL` for the worker must be <https://api.bulsuscholar.com>.
- [ ] Use production-only random secrets. The API and waitlist worker must share `CRON_SECRET`; OTP and recovery secrets must be different from each other and from staging.
- [ ] Deploy Latest Commit for the approved SHA. Do not use Redeploy on an older deployment. Wait for Success and inspect logs.
- [ ] Run: `Invoke-RestMethod https://api.bulsuscholar.com/health` and `/deployment/health`. Load `/openapi.json` and confirm auth, email-verification, profile/document, roster, scope/report, announcement, waitlist, recovery, history, and Signed SOE routes.
- [ ] Update/deploy waitlist-expiry from the same SHA, run it once, confirm `ok=true`, then verify its `* / 5 * * * *` schedule.
- [ ] From the repository with `BACKEND_URL` configured, run `npm.cmd run verify:deployment` and save the result.
- [ ] Gate: production API and cron are healthy on the approved SHA before the production frontend is promoted.

## Phase 11 - Deploy the production Vercel frontend

- [ ] In Vercel Environment Variables -> Production, verify the public production values: Supabase URL/anon key/bucket, <https://bulsuscholar.com> for app/public URLs, and <https://api.bulsuscholar.com> for backend/document scan URLs.
- [ ] Confirm no Railway-only secret is present. Vite embeds VITE\_ values into the public browser bundle.
- [ ] Promote/deploy the approved SHA to Production. Confirm Ready, the production domain assignment, and the same Git commit shown by Railway.
- [ ] Open <https://bulsuscholar.com> in a private window. Maintenance Mode should still appear. Check console/network output and verify requests use the production API, not localhost or staging.
- [ ] Gate: Vercel and Railway run the same approved SHA, domains/TLS work, and Maintenance Mode still protects normal workflows.

## Phase 12 - Production smoke test, release, and monitoring

- [ ] While Maintenance Mode is on, verify root health/settings, production API health, cron status, Supabase readiness/denials, Storage authorization, and one controlled Brevo delivery.
- [ ] Schedule a short controlled opening. Turn Maintenance Mode off and immediately test one disposable student, grantor, full admin, and root login plus dashboard/API bootstrap.
- [ ] Run a narrow smoke path: pending approval, private document preview, one ordinary application, one roster state, one announcement/notification, one support/recovery check, and waitlist status without disturbing real awards.
- [ ] Check Railway logs, Vercel/runtime/browser errors, Supabase Auth/database logs, Brevo transactional logs, notification counts, and cron execution for at least the agreed observation window.
- [ ] If a core/security issue appears, immediately re-enable Maintenance Mode. Roll back Vercel and Railway to the recorded working deployment only when database compatibility permits it.
- [ ] Do not attempt to reverse database migrations casually. Use an explicit corrective migration, or restore the recorded backup only after assessing all post-backup writes and Storage differences.
- [ ] When every gate passes, record release owner, date/time, approved SHA, migration log, backup IDs, Railway/Vercel deployment IDs, tester sign-offs, known nonblocking issues, and monitoring owner.

## 6. Johnvher - Admin Test Suite

---

Role: full admin. Run on a disposable staging environment using desktop 1440 x 900. Do not use real student records for destructive tests.

### ADM-01 - Pending account approval and rejection

**Preconditions:** Two email-confirmed pending students and access to the test mailbox.

**Steps:**

1. Open Admin -> Student Management -> Pending.
2. Inspect the first student's information and accept the account.
3. Confirm the student moves to active records and receives one Welcome email.
4. Reject the second student with a safe reason.
5. Confirm one Rejected email, restricted decision history, and clean restart behavior.

**Expected result:** Only a full admin can decide. No pending account can log in before acceptance, and no duplicate email/notification is created.

### ADM-02 - Document review and ROG rules

**Preconditions:** One first-year first-semester student and one upper-year student with pending COR/ROG/identity/profile versions.

**Steps:**

1. Open Document Review and filter by cycle and status.
2. Confirm the first-year first-semester student's ROG says Not required for this semester.
3. Approve COR, allowed identity document, and profile revision.
4. For the upper-year student, reject ROG with a reason, then approve a corrected version.
5. Confirm tracking pauses at the first incomplete requirement and restores later progress after approval.

**Expected result:** ROG exemption is server-derived only for first-year first-semester. Rejection and correction history remain visible and immutable.

### ADM-03 - Grantor classification and location scope

**Preconditions:** A disposable grantor and two municipalities.

**Steps:**

1. Set the grantor classification to Government, Private, or Others.
2. Enable a named scope containing one municipality and save.
3. Verify an in-scope new applicant can proceed and an out-of-scope applicant receives a clear denial.
4. Change the scope and confirm an existing application keeps its earlier scope snapshot.

**Expected result:** Scope affects new admissions only, uses the self-declared permanent address, and cannot be bypassed by changing request IDs.

## ADM-04 - Roster preview, conflict, and idempotency

**Preconditions:** Roster file containing valid, duplicate, unknown-scholarship, name-mismatch, and commitment-conflict rows.

**Steps:**

1. Upload/map the file and generate the server preview.
2. Confirm blocking rows show specific reasons and Commit is disabled.
3. Correct the file and preview again.
4. Commit the valid batch, then attempt to commit the same batch again.
5. Resolve one conflict only after identity verification and select the correct roster row.

**Expected result:** No duplicate award is created. Matching applications retain IDs/history/documents, public slots are released, and conflict alternatives archive without cooldown.

## ADM-05 - Student-number correction

**Preconditions:** Disposable active student with applications, documents, notifications, and history.

**Steps:**

1. Start correction, enter a unique new number, confirmation, and audit reason.
2. Confirm the Auth UUID and linked history remain unchanged.
3. Verify the old number no longer logs in and existing sessions are revoked.
4. Log in with the new number and confirm all owned records remain accessible.

**Expected result:** The update is atomic/audited and does not orphan documents, applications, tickets, or history.

## ADM-06 - Targeted announcement and deduplication

**Preconditions:** At least three active students in different course/year groups.

**Steps:**

1. Create an announcement and choose a permitted audience filter.
2. Generate the audience preview and record recipient count.
3. Publish from the preview, then retry the same publish action.
4. Check the intended students' dashboard and inbox; check a nonrecipient account.

**Expected result:** Each intended recipient gets one delivery and correct direct link; nonrecipients get none; expired previews cannot publish.

### ADM-07 - Signed SOE review and reopen

**Preconditions:** Ordinary committed student whose Signed SOE stage is Submitted.

**Steps:**

1. Open Requirements/SOE review and preview the private file.
2. Reopen without a reason and confirm the action is blocked.
3. Enter a reason and reopen.
4. Verify the student returns to Upload Signed SOE and receives one correction notice.
5. After replacement upload, confirm both immutable versions remain.

**Expected result:** Reopen requires Requirements permission and a reason; the system never labels the file staff-verified merely because it was uploaded.

### ADM-08 - Permission and interface checks

**Preconditions:** One full admin and one limited/reviewer admin.

**Steps:**

1. Attempt each sensitive action with the limited account.
2. Confirm hidden/disabled controls match backend denials.
3. Check dialogs with keyboard Tab, Shift+Tab, Escape, and focus restoration.
4. Check light/dark mode, long names, empty/loading/error states, toasts, and image previews.

**Expected result:** Backend authorization remains decisive; no clipping, unreadable contrast, modal layering error, duplicate toast, console error, or failed request remains.

[ ] Johnvher sign-off: all ADM cases passed and evidence links are recorded.

Tester signature: \_\_\_\_\_ Date/time: \_\_\_\_\_ Build/commit: \_\_\_\_\_

## 7. Veejay - Grantor Test Suite

Role: grantor. Run at 1440 x 900 and 390 x 844 in light and dark modes. Use only the grantor's own scholarships and disposable applicants.

### GRA-01 - Login, email verification, and tab behavior

**Preconditions:** Grantor account with valid email; activity timestamp tested at 29 days and at least 30 days.

**Steps:**

1. At 29 days, sign in with the correct password and confirm normal dashboard access.
2. At 30 days, confirm password success leads to Email verification without exposing a portal session.
3. Test wrong code attempts, 60-second resend delay, old-code invalidation, and successful code.
4. Open a second normal tab and confirm it shares the account; log out and confirm both tabs exit.

**Expected result:** OTP appears only when due, limits are enforced, no token is released before verification, and Incognito remains independent.

### GRA-02 - Scholarship announcement, image, slots, and audience

**Preconditions:** One recognized scholarship program and owned applicant/scholar fixtures.

**Steps:**

1. Create an application announcement with image, exact scholarship, schedule, and slots.
2. Preview a grantor-owned audience and publish.
3. Verify the announcement card image appears consistently in dashboard, announcement detail, and recommendation card.
4. Confirm an unrelated student/grantor is not included.

**Expected result:** Published content, image, slot count, audience, inbox notice, and direct link are correct with one delivery per student.

### GRA-03 - Roster preview and commit

**Preconditions:** Spreadsheet with one future-account student, one matching applicant, one duplicate, and one unknown scholarship.

**Steps:**

1. Upload and map the spreadsheet.
2. Review server row statuses and exact matched scholarship.
3. Confirm Commit remains disabled while blocking errors exist.
4. Correct and re-preview; commit once; repeat the commit.
5. Check the converted applicant ID/history and the future-account No account yet label.

**Expected result:** The batch is idempotent, no duplicate award is created, converted history is preserved, and roster awards consume no public slot.

## GRA-04 - Applicant filters and reports

**Preconditions:** Applicants across statuses, municipalities, scholarships, cycles, document states, and tracking stages.

**Steps:**

1. Apply each filter individually and in combination.
2. Export CSV and PDF and compare displayed/total counts.
3. Attempt to substitute another grantor ID in the request.
4. Inspect report audit information as authorized staff.

**Expected result:** Only the grantor's own records appear. Cross-grantor access is denied and exports are generated from backend records, not browser-supplied rows.

## GRA-05 - Decision, slot return, and waitlist promotion

**Preconditions:** A full scholarship with queued students and one disposable active applicant.

**Steps:**

1. Reject or archive the disposable application using the allowed workflow.
2. Confirm the public slot returns exactly once.
3. Verify the first eligible queued student receives an exclusive 24-hour offer.
4. Add slots and confirm FIFO promotion continues.
5. Verify duplicate actions do not create duplicate offers or notifications.

**Expected result:** Slot accounting remains consistent, queue order is FIFO, and only eligible students receive one actionable notice.

## GRA-06 - Document visibility and authority boundary

**Preconditions:** Owned applicant with approved and pending private documents.

**Steps:**

1. Open documents related to the owned application.
2. Confirm approved/application-relevant records can be viewed through authorized short-lived access.
3. Attempt to approve/reject a student-level document.
4. Attempt to access another grantor's applicant/document URL.

**Expected result:** Grantor can view only relevant permitted files and cannot issue authoritative document decisions or cross grantor boundaries.

## GRA-07 - Mobile and desktop interface

**Preconditions:** Realistic long names, empty list, many records, modal, toast, archived state, and inbox fixtures.

**Steps:**

1. Check Dashboard, Scholars, Applications, Announcements, Inbox, and Profile at both viewports.
2. Open composer, archive/reject confirmations, image preview, document preview, and inbox detail.
3. Use keyboard and touch-size controls; switch light/dark mode.
4. Check console/network errors and horizontal overflow.

**Expected result:** No clipped text, inaccessible close control, hidden focus, overlay-behind-modal error, full-width action failure, duplicate toast, or unexpected failed request remains.

[ ] Veejay sign-off: all GRA cases passed and evidence links are recorded.

Tester signature: \_\_\_\_\_ Date/time: \_\_\_\_\_ Build/commit: \_\_\_\_\_

## 8. Emerson - Student Test Suite

Role: student. Run at 1440 x 900 and 390 x 844 in light and dark modes. Use disposable documents and applications.

### STU-01 - Signup, confirmation, and approval

**Preconditions:** Unused student ID/email and access to the test mailbox.

**Steps:**

1. Complete signup and confirm the email.
2. Attempt login before admin approval.
3. Have Johnvher approve the pending account.
4. Verify the Welcome email and successful login.

**Expected result:** The account stays pending until approved, cannot log in early, and becomes active only after the committed admin decision.

### STU-02 - Profile draft, signature, PDF, and document requirements

**Preconditions:** Approved first-year first-semester student, then an upper-year/cycle fixture.

**Steps:**

1. Complete permanent address, leave current address optional, save and reopen the draft.
2. Draw/upload a fresh signature, submit, and inspect the official PDF.
3. Upload COR, allowed identity document, and verify ROG shows Not required for first-year first-semester.
4. Repeat with upper-year/second-semester data and confirm ROG becomes required.

**Expected result:** Submitted revisions are immutable, drafts remain editable, files are private, and requirement decisions come from the server.

### STU-03 - Multiple applications and one commitment

**Preconditions:** Two open eligible scholarships from different grantors with available slots.

**Steps:**

1. Apply to both scholarships and confirm both current applications remain visible.
2. Use the Scroll Stack controls, wheel/swipe, keyboard arrows, and progress dots.
3. Complete requirements for one and request materials.
4. Confirm the selected scholarship commits and the alternative moves to history with its slot released.

**Expected result:** Multiple applications are allowed before commitment; only one scholarship remains after confirmation and controls remain usable at both viewports.

### STU-04 - Request Materials rejection and retry

**Preconditions:** Ordinary application with one missing/rejected current document, then a corrected approved version.

**Steps:**

1. Request materials while the requirement is incomplete.
2. Record the specific displayed reason and confirm no partial state changed.
3. Upload correction and have it approved.
4. Double-click Request Materials or submit concurrently, then retry normally.

**Expected result:** The first request fails precisely and atomically; corrected retry succeeds once with no duplicate commitment/request.

### STU-05 - FIFO waitlist offer

**Preconditions:** Full scholarship, waitlist enabled, Emmerson queued first, another student queued second.

**Steps:**

1. Join the queue and record displayed position.
2. Release/add one slot and verify Emmerson receives one inbox/dashboard 24-hour offer.
3. Accept before expiry and confirm an ordinary application is created, not an award.
4. In a second run, decline/expire and confirm the next eligible student is offered the slot.

**Expected result:** FIFO position and reserved offer are accurate; accept/decline/expiry is atomic; rejoin goes to the queue end.

### STU-06 - Authoritative roster lock and archival

**Preconditions:** Exactly one valid active roster match and approved account.

**Steps:**

1. Confirm automatic authoritative assignment after approval.
2. Verify the required identity documents remain visible until approved.
3. Attempt withdrawal and a competing application while the roster row is active.
4. Archive the individual roster row through staff, wait/check the 24-hour cooldown, then verify later application eligibility.

**Expected result:** Roster scholar cannot withdraw/change while active, uses no public slot, and becomes eligible only after individual archival and cooldown.

### STU-07 - Tracking pause and recovery

**Preconditions:** Tracking previously progressed beyond document review, then one earlier document becomes invalid.

**Steps:**

1. Confirm the first incomplete earlier step becomes On-going and every later step appears gray/pending.
2. Verify later completion records are not deleted.
3. Submit and obtain approval for the corrected document.
4. Confirm the tracker restores the previously active stage automatically.

**Expected result:** Sequential display is correct and preserved progress returns without duplicate tracking history.

## STU-08 - Signed SOE, Finish, and History

**Preconditions:** Ordinary committed application with office signing complete.

**Steps:**

1. Try wrong type, oversized file, wrong stage, and duplicate submission.
2. Upload one valid PDF/PNG/JPEG up to 10 MB.
3. Confirm Upload Signed SOE shows Submitted, Finish completes, and the renewal message appears.
4. Open History, filter by cycle/type, and follow the related link.
5. After admin reopen, upload a corrected version and confirm both events/versions remain.

**Expected result:** A valid upload completes the cycle immediately as submitted, not verified; history is chronological and contains no staff-only notes.

## STU-09 - Email verification and lost-email recovery

**Preconditions:** One 30-day-inactive student and one signed-out lost-email fixture.

**Steps:**

1. Complete password login and verify the six-digit challenge blocks session release.
2. Test expiry, three wrong codes, resend delay, old-code invalidation, and successful verification.
3. From Help, create a Lost email access ticket and retain the secret link.
4. Attach valid private evidence; confirm a tampered link cannot read it.
5. After root approval, confirm the proposed mailbox code and verify old sessions are revoked.

**Expected result:** The flow reveals no account existence, changes no email before mailbox confirmation, and keeps evidence private.

## STU-10 - Student UI, inbox, notifications, and accessibility

**Preconditions:** Announcements, unread notices, long text, empty/loading/error states, and one image preview.

**Steps:**

1. Check Dashboard, Announcements/detail, Inbox/detail, Profile, Scholarships, Recommendations, and History on desktop/mobile.
2. Verify Quick Actions use full available width, Support opens Help, announcement cards are consistent, and View Announcement is full width where required.
3. Check light/dark contrast, toasts, modal Escape/backdrop behavior, keyboard order, image preview layering, and icon labels.
4. Confirm workflow notices arrive once and direct links open the intended section.

**Expected result:** No clipping, horizontal overflow, unreadable text, duplicate notification/toast, inaccessible icon control, modal layering error, console error, or failed request remains.

[ ] Emmerson sign-off: all STU cases passed and evidence links are recorded.

Tester signature: \_\_\_\_\_ Date/time: \_\_\_\_\_ Build/commit: \_\_\_\_\_

## 9. Role Test Sign-Off

| Gate                  | Owner            | Required evidence                                                      | Pass |
|-----------------------|------------------|------------------------------------------------------------------------|------|
| Backups and rollback  | Release owner    | Database restore point, Storage copy/manifest, rollback commit         | [ ]  |
| Database and RLS      | Backend/DB owner | Migration log, advisors, anon/authenticated denial checks              | [ ]  |
| Backend and cron      | Railway owner    | Healthy deployment, OpenAPI paths, cron ok=true                        | [ ]  |
| Frontend              | Vercel owner     | Same commit Ready, correct API URL, no console errors                  | [ ]  |
| Admin acceptance      | Johnvher         | ADM-01 through ADM-08                                                  | [ ]  |
| Grantor acceptance    | Veejay           | GRA-01 through GRA-07                                                  | [ ]  |
| Student acceptance    | Emmerson         | STU-01 through STU-10                                                  | [ ]  |
| Root acceptance       | Root admin       | Recovery, admin unblock, waitlist capacity, audit, destructive dialogs | [ ]  |
| Email and Storage     | Release owner    | Brevo deliveries and private file authorization                        | [ ]  |
| Production smoke test | All owners       | Role flows, timestamps, screenshots, issue/retest record               | [ ]  |

### Pre-release status

These role and infrastructure gates feed the final September 18-present verification checklist at the end of this guide. Number 6 remains partially blocked until the scholarship office supplies and approves the official point formula.

Release owner: \_\_\_\_\_

Approved commit: \_\_\_\_\_

Database backup/restore point: \_\_\_\_\_

Deployment date/time: \_\_\_\_\_

Final decision: [ ] Release [ ] Keep Maintenance Mode enabled

Notes / unresolved issues:

## 10. Complete Change Register: September 18-Present

### Comparison basis

Baseline commit: 97383b0 (Added Support Conversation), pushed September 17, 2026. Current comparison includes the September 18 and September 22 pushes plus the uncommitted working tree through September 28, 2026.

### Git status warning

The latest GitHub push is 1752c0f from September 22. The Numbers 4-10 implementation, five later migrations, UI audit, this PDF, and related backend/frontend files are still modified or untracked locally. They are not on GitHub until reviewed, committed, and pushed.

### Measured tracked-file impact

- 82 tracked files changed relative to the September 17 baseline.
- 6,429 tracked-line additions and 3,974 tracked-line deletions.
- 25 additional untracked files currently span backend services/tests, frontend services/pages/components, documentation/scripts, and Supabase migrations.
- Three recorded pushes in this window: September 18 at 10:16 AM, September 22 at 3:32 PM, and September 22 at 10:30 PM.

### A. Change timeline

| Date/state          | Change group                           | System effect                                                                                                                                                                                                                                                                  |
|---------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sep 18 push 9ffca0  | Scholarship document validation        | Expanded scholarship readiness beyond COR/ROG to include Student ID and Student Application Profile, accepted legacy string URLs, improved eligibility/correction messages, and reduced unnecessary console errors for expected ineligibility.                                 |
| Sep 22 push 7705a43 | Tracking, roster, and login foundation | Added compliance-pause tracking, Finish stage, automatic roster/login-security migration, unified backend Auth, failed-login tracking/recovery, grantor Auth migration, portal session checks, and removal of the browser-side roster-choice gate and legacy password service. |
| Sep 22 push 1752c0f | Security cutover and account approval  | Added unsafe-access revocation, manual student approval records/endpoints, pending-session rejection, full-admin account review, Storage backup/private-bucket scripts, tests, and security cutover documentation.                                                             |
| Sep 23-28 local     | Numbers 4-10 implementation            | Added profile/document review, roster preview/commit/conflicts, scope/reporting, targeted announcements, waitlist, OTP/session/recovery, history/Signed SOE, portal compatibility gateway, UI changes, release tests, migrations, and acceptance documentation.                |

### B. Logic, data, and security changes

- Sensitive portal reads/writes moved behind authenticated backend routes with role, actor, record-owner, and grantor-owner checks. Student roster summaries redact other students' identity data.
- Anonymous/authenticated direct table privileges were revoked for protected records; the Storage bucket was made private and private content is served only after authorization.
- Student signup validates and finalizes server-side, confirms email, enters a Pending review queue, and cannot bootstrap a portal session until a full admin accepts the account.
- Backend login now resolves User ID safely, checks blocked/disabled state, uses Supabase Auth, records concurrent-safe failures, locks at the configured limit, and resets failures on successful authentication/recovery.

- Thirty-day student/grantor inactivity now triggers a hashed six-digit email challenge before session tokens are released. Verified sessions bind Auth UUID, session ID, account identity, lock state, and sessionValidAfter.
- Normal tabs share portal identity and logout through local storage/BroadcastChannel; Incognito and separate devices retain independent sessions.
- Lost-email recovery now uses secret-capability support conversations, private attachments, root review, proposed-email code confirmation, atomic Auth/profile update, failure reset, audit, and old-session revocation.
- Profile handling changed from a file-only model to editable drafts, fresh signatures, immutable submitted revisions, generated official PDFs, private Storage, per-application snapshots, and revision history.
- COR, ROG, identity, and profile submissions are immutable versions with pending/approved/rejected decisions. First-year first-semester ROG exemption is server-derived; later semesters/years require ROG.
- COR policy supports cor\_only, advising\_only, or either; individual exceptions are reasoned and audited. Scanner classification does not equal approval.
- Tracking pauses at the first incomplete requirement. Later steps turn gray but remain preserved in compliance-pause metadata; corrected approval restores the previous active stage.
- Ordinary applications may coexist before commitment. Request Materials is atomic and validates ownership, account, eligibility, approved document snapshot/versions, application requirements, slot, roster conflicts, cooldown, and existing commitment.
- Authoritative roster import now requires server Preview then Commit, exact recognized scholarship ownership, stable batch fingerprints, row classifications, idempotency, advisory locking, in-place application conversion, and visible conflict resolution.
- Roster awards consume no public slot and fabricate no files/events. Active individual roster membership locks withdrawal/competing applications; individual archive releases the commitment and starts the 24-hour cooldown; grantor archive transfers servicing to admin.
- Grantor classification and versioned municipality scope are separate from provider type. New application/waitlist decisions snapshot the self-declared permanent address and applied scope version.
- Applicant filtering and CSV/PDF exports are backend-owned and audited. Full-admin student-number correction preserves Auth UUID/history, updates live references, changes Auth metadata, revokes sessions, and records a retryable audit state.
- Announcement audience selection uses a 15-minute server preview and immutable recipient snapshot. Deterministic delivery IDs prevent duplicate inbox/dashboard notices.
- A root-controlled global waitlist cap defaults to zero. Per-scholarship FIFO queues create exclusive 24-hour slot offers; accept creates an ordinary application, decline/expiry advances the queue, and commitment closes competing entries/offers.
- The protected waitlist expiry endpoint uses CRON\_SECRET and is designed for a five-minute Railway cron worker.
- Student history is a private append-only deterministic ledger covering application, document, tracking, roster, waitlist, commitment, cooldown, withdrawal, and cycle events without staff-only notes.
- Ordinary committed scholars now upload Signed SOE after office signing and before Finish. Valid PDF/PNG/JPEG submission completes the cycle as Submitted; authorized admins can reopen with a reason while preserving all versions. Roster scholars are exempt.
- The official ranking formula remains intentionally unimplemented. Existing output is labelled Recommendation Score - Not Official Ranking.

## C. User-interface changes

- Replaced wording-based runtime button classification with explicit positive, danger, neutral, and unstyled semantics; destructive color is consistent and links/rows/icon controls remain unboxed.
- Added shared Light mode/Dark mode switch behavior and dark-mode contrast corrections; loading overlay uses the darker green backdrop.

- Student inbox rows are borderless clickable surfaces; message detail uses one top-right X; delete remains a compact danger icon; footer Quick Links are navigation links.
- Back to Dashboard uses neutral styling and arrow-left; View Announcement includes an eye icon and full-width placement; the duplicate Recommended Scholarships header action was removed.
- Student dashboard Quick Actions use equal desktop columns and responsive mobile stacking; Support opens Help and Support; the History shortcut was added.
- Announcement cards were aligned across student dashboard/list/detail/recommendation surfaces, including preserved scholarship images and links to the correct announcement detail.
- Multiple current applications use the contained Motion Scroll Stack with wheel, swipe, previous/next, arrows, progress dots, stable selection, reduced motion, and one interactive front card. Previous applications are separated from current records.
- Student Profile now exposes draft completeness, Save Draft, Preview, Submit for Review, signature input, document-version status, rejection reasons, correction actions, and first-year ROG exemption copy.
- Admin gained Pending Accounts, Document Review, roster conflicts/import preview, grantor scope/classification, applicant reports, student-number correction, targeted announcement preview, Signed SOE review/reopen, and permission-denied states.
- Grantor gained server roster preview/commit results, scoped applicant filters/exports, announcement audiences, location information, waitlist-related slot behavior, and refined mobile announcement/profile layouts.
- Login added Email verification UI and improved mobile form-first layout. Help added signed-out Lost email access conversation and attachment controls.
- Student History page provides filters, pagination, safe descriptions, and related links. Scholarship tracking includes Upload Signed SOE and Finish/renewal states.
- Shared modal accessibility now provides dialog semantics, initial focus, Tab trapping, Escape close, focus restoration, accessible icon labels/tooltips, and corrected nested image-preview layering.
- UI audit fixed the Admin Profile missing-icon runtime crash, student dark-mode empty-state readability, grantor mobile header/save clipping, signup preview close icon, and five-column Quick Actions layout.
- Recommendation wording no longer presents the informational score as an official ranking.

## D. Operational and development-process changes

- Added additive migrations for compliance preservation, Finish, automatic roster/login security, access revocation, manual approval, profile/document review, roster workflow, scope/reporting/announcements/waitlist, OTP/history/Signed SOE, and complete history backfill.
- Added a dry-run-first idempotent grantor-to-Supabase-Auth migration script and removed browser-side legacy password encryption/reset utilities.
- Added Storage backup/private-bucket scripts, a security cutover record, a beginner deployment checklist, the tapos build/commit/push helper, waitlist expiry worker, and frontend security regression script.
- Deployment order is now explicit: Maintenance Mode and backups -> staging migrations -> database compatibility -> backend -> health/OpenAPI/cron -> frontend -> role acceptance -> production smoke test -> reopen.
- Required backend-only secrets now include separate portal email code, public recovery code, cron, root session, service-role, Brevo, frontend URL, and private Storage configuration. Server secrets must never be exposed as VITE variables.
- Local release evidence currently includes 136 backend tests, six frontend security checks, ESLint, production build, backend compilation, offline parsing of all migrations, and desktop/mobile browser/UI checks.
- Production acceptance still requires real Supabase transaction/RLS/Storage tests, Brevo delivery, Auth session behavior, Railway cron, concurrency/retry, authenticated role workflows, and rollback evidence.

## E. New migrations introduced in this change window

| Version        | Purpose                                       |
|----------------|-----------------------------------------------|
| 20260918120000 | Preserve tracking during document compliance  |
| 20260918123000 | Add tracking Finish stage                     |
| 20260918130000 | Automatic roster and login security           |
| 20260922170000 | Revoke unsafe direct access                   |
| 20260922180000 | Require manual student account approval       |
| 20260923120000 | Student profile and document verification     |
| 20260923124643 | Scholarship roster workflow                   |
| 20260924024322 | Scope, reporting, announcements, and waitlist |
| 20260924125213 | Security, history, and Signed SOE             |
| 20260925103000 | Complete student history ledger               |

## 11. Consolidated Tests for September 18-Present

Run these checks in staging against the exact commit intended for deployment. The Johnvher, Veejay, and Emmerson suites remain the detailed role procedures; this section is the change-to-test traceability list.

| Change area                   | Required test evidence                                                                                                                                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Access containment            | Anon/authenticated direct reads, writes, deletes, privileged RPCs, Realtime subscriptions, and private file URLs fail; permitted backend-owned role reads succeed.                                           |
| Signup approval               | Confirmed signup stays Pending; pending login fails; full-admin approve/reject works; Welcome/Rejected email is single; rejection restart and cleanup are safe.                                              |
| Login logout/recovery         | Limits 3 and 10, simultaneous failures, successful reset, blocked sessions, generic unknown-ID response, student/grantor recovery, root-only admin unblock.                                                  |
| Thirty-day email verification | 29/30-day boundary, no token before code, 10-minute expiry, three wrong codes, 60-second resend, five sends/hour, replay, Brevo failure/retry.                                                               |
| Session behavior              | Normal-tab sharing/logout, Incognito and second-device independence, token refresh, deleted Auth session, lock/password reset revocation, stale JWT denial.                                                  |
| Profile/revisions             | Draft save/reopen, validation, fresh signature, immutable revision, one-page PDF, application snapshot, history, permanent/current address behavior.                                                         |
| Document rules                | First-year first-semester ROG exemption, later ROG requirement, COR modes/exception, first-year photo-ID alternatives, upper-year Student ID/lost-card exception, scanner not approval.                      |
| Review/tracking recovery      | Pending/approve/reject/resubmit, reason required, compliance pause, later steps gray, preserved completions, corrected approval restores prior active stage without duplicate history.                       |
| Roster import                 | Unknown/ambiguous scholarship, duplicates, name mismatch, future account, matching application conversion, other roster/commitment conflict, repeat/concurrent commit, no public-slot use.                   |
| Roster lifecycle              | Active lock, document-gated system Finish, individual archive and 24-hour cooldown, full grantor archive to admin servicing, no fabricated documents/material/signature/SOE.                                 |
| Ordinary application          | Multiple precommit applications, Scroll Stack controls, exact eligibility/scope/cooldown/slot checks, Request Materials rejection atomicity, duplicate/concurrent success, alternative closure/slot release. |
| Scope and reports             | Scope on/off, inside/outside normalized address, version snapshot, cross-grantor denial, filters/pagination/counts, CSV/PDF audit, student-number correction and session revocation.                         |
| Announcement delivery         | Every audience type, 15-minute expiry, preview count, grantor ownership, retry deduplication, disabled/archived recipients, unread counts, dashboard/inbox direct links.                                     |
| Waitlist                      | Disabled/global cap, FIFO ties, simultaneous joins, release/add-slot promotion, reserved offer, accept/decline/expiry/rejoin, no eligible successor, commitment cleanup, cron secret/schedule.               |
| Lost-email recovery           | Unknown User ID privacy, capability tamper, attachment type/size/count/private access, root reject/approve, duplicate email, proposed-mailbox code, partial Auth retry, old-session revocation.              |
| History                       | Backfill count and deterministic deduplication, chronological pagination, filters, related links, cycle preservation, absence of reviewer/security notes.                                                    |
| Signed SOE                    | Wrong owner/stage/cycle/type/size/duplicate, immediate Submitted/Finish, renewal copy, authorized preview, reason-required reopen, replacement version, roster exemption.                                    |
| UI and accessibility          | Student/grantor desktop+mobile and Admin/root desktop; light/dark; empty/loading/error/conflict/archived states; toasts; modals; focus; Escape; labels; preview layering; no overflow/console/network error. |
| Deployment                    | Backup restore point and Storage manifest, migration log, RLS/advisors, matching Railway/Vercel commit, health/OpenAPI, cron ok=true, rollback rehearsal, role smoke tests.                                  |

### Recommended execution order

- [ ] Freeze the staging dataset and record fixture IDs, application IDs, queue positions, notification counts, current commit, and migration state.
- [ ] Run automated backend/frontend/build checks and preserve their output.
- [ ] Run RLS, direct-access, private Storage, and cross-role authorization checks before functional testing.
- [ ] Run Johnvher's account, document, roster, scope, announcement, student-number, and Signed SOE scenarios.
- [ ] Run Veejay's login, announcement, roster import, applicant report, slot/waitlist, document-boundary, and responsive scenarios.
- [ ] Run Emmerson's signup, profile/documents, multiple applications, materials, waitlist, roster lock, tracking recovery, Signed SOE, OTP/recovery, and responsive scenarios.
- [ ] Run root-only recovery review, admin unblock, waitlist capacity, audit, settings, and destructive confirmations.
- [ ] Run concurrency and retry checks for login failures, roster commits, Request Materials, waitlist joins/promotions, audience publishing, emails, and Storage operations.
- [ ] Check database state, history/audit rows, notifications, emails, and UI result after every destructive transition.
- [ ] Retest each defect after correction and retain before/after evidence.
- [ ] Run the production smoke test under Maintenance Mode or a controlled window, then make the release decision.

**Final release rule**

Do not deploy the current workspace until all intended modified and untracked files are reviewed and committed, the same commit is deployed to Railway and Vercel, all required migrations are confirmed on staging, and every release-blocking test above passes. Keep Maintenance Mode enabled when any result is missing or unclear.

September 18-present test owner: \_\_\_\_\_

Approved commit containing local revisions: \_\_\_\_\_

Evidence location: \_\_\_\_\_

Final result: [ ] Pass for deployment [ ] Blocked / retest required